

C++ std::string 完整用法总结

std::string 是 C++ 标准库中的一个类，用于表示和操作动态大小的字符串。以下是 std::string 的常见初始化方法、成员函数、运算符重载等相关用法的详细总结。

1. 初始化 std::string

std::string 提供了多种初始化方式：

- 默认构造：

```
std::string str1; // 创建一个空字符串
```

- 通过 C 风格字符串初始化：

```
std::string str2 = "Hello"; // 使用字符串字面量初始化
```

- 指定字符数量和字符初始化：

```
std::string str3(10, 'A'); // 创建一个包含10个字符 'A' 的字符串
```

- 通过子字符串构造：

```
std::string str4(str2, 1, 3); // 从 str2 中索引 1 开始，提取 3 个字符
```

- 通过另一个 std::string 初始化（复制构造）：

```
std::string str5 = str2; // 创建一个新的字符串，复制 str2 的内容
```

- 通过 C 风格字符数组构造：

```
const char* cstr = "World";  
std::string str6(cstr); // 从 C 风格字符串初始化
```

- 通过迭代器范围初始化：

```
std::string str7(begin(str2), end(str2)); // 使用迭代器范围构造字符串
```

- 移动构造 (C++11)：

```
std::string str8 = std::move(str5); // 通过移动构造，避免数据复制
```

2. 常见成员函数

2.1 字符串信息查询

- size() / length()：返回字符串的长度。

```
size_t len = str2.size(); // 等同于 length()
```

- empty()：判断字符串是否为空。

```
if (str2.empty()) { std::cout << "String is empty!" << std::endl; }
```

- capacity()：获取字符串当前的容量（实际分配的内存大小）。

```
size_t cap = str2.capacity();
```

- **reserve()** : 为字符串预留足够的空间, 避免多次内存分配。

```
str2.reserve(100); // 为 str2 预留 100 个字符的空间
```

- **resize()** : 改变字符串的大小。

```
str2.resize(15, 'X'); // 将字符串大小调整为 15, 不足部分填充字符 'X'
```

2.2 字符串操作

- **append()** : 在字符串末尾追加内容。

```
str2.append(" world!"); // 添加字符串到末尾
```

- **push_back()** : 向字符串末尾添加单个字符。

```
str2.push_back('!');
```

- **insert()** : 在指定位置插入子字符串。

```
str2.insert(5, ", "); // 在索引 5 处插入 ", "
```

- **erase()** : 删除指定位置的字符或子字符串。

```
str2.erase(5, 2); // 删除索引 5 开始的 2 个字符
```

- **replace()** : 替换指定位置的字符或子字符串。

```
str2.replace(6, 5, "C++"); // 将从索引 6 开始的 5 个字符替换为 "C++"
```

- **substr()** : 提取子字符串。

```
std::string sub = str2.substr(0, 5); // 提取从索引 0 开始的 5 个字符
```

- **find()** : 查找子字符串首次出现的位置。

```
size_t pos = str2.find("world"); // 查找 "world" 在 str2 中的位置
```

- **rfind()** : 从右向左查找子字符串。

```
size_t pos = str2.rfind("world");
```

- **compare()** : 字符串比较。

```
int result = str2.compare("Hello, C++!"); // 返回 0 表示相等, 负数表示小于, 正数表示大于
```

2.3 C 风格字符串接口

- **c_str()** : 返回一个 `const char*` 类型的 C 风格字符串。

```
const char* cstr = str2.c_str();
```

- **data()** : 返回指向字符数组的指针 (与 `c_str()` 相似, 但在某些实现中返回的指针可用于修改字符内容)。

```
char* data = str2.data();
```

3. 常见运算符重载

`std::string` 支持许多常见的运算符，方便我们进行字符串操作：

- `+` / `+=` : 字符串连接。

```
std::string str3 = str2 + " C++"; // 使用 + 连接
str2 += " is awesome"; // 使用 += 连接
```

- `[]` : 按字符索引访问字符。

```
char ch = str2[0]; // 获取字符串的第一个字符
```

- `==` / `!=` : 字符串比较是否相等或不相等。

```
if (str2 == "Hello, C++!") { std::cout << "Equal!" << std::endl; }
if (str2 != "Hello") { std::cout << "Not equal!" << std::endl; }
```

- `<`, `>`, `<=`, `>=` : 字符串字典顺序比较。

```
if (str2 < str3) { std::cout << "str2 is lexicographically less than str3" << std::endl; }
```

4. 现代C++特性

4.1 移动语义 (C++11)

`std::string` 在 C++11 中支持了移动语义，使得字符串在赋值或传递时可以避免不必要的内存复制，从而提高性能：

```
std::string str4 = std::move(str2); // 通过移动构造，避免数据复制
```

4.2 范围-based for 循环 (C++11)

可以直接使用范围-based for 循环遍历 `std::string` 的字符：

```
for (char ch : str2) {
    std::cout << ch << " ";
}
```

4.3 初始化列表与字符串构造

通过初始化列表和 `std::initializer_list` 可以方便地创建字符串：

```
std::string str5 = {'H', 'e', 'l', 'l', 'o'}; // 通过初始化列表构造
```

4.4 `std::move` 和 `std::string` 的移动赋值

在 C++11 及之后的版本，`std::string` 支持移动赋值，避免了不必要的字符串复制：

```
std::string str6 = "Move this";
std::string str7 = std::move(str6); // 移动语义，str6 被清空
```

5. 总结

- `std::string` 是现代 C++ 中处理字符串的核心类，提供了丰富的初始化方式、成员函数、运算符重载及高效的内存管理。
- 利用 C++11 引入的特性，如移动语义、范围-based for 循环等，`std::string` 提供了更高效、简洁的字符串操作方法。

- `std::string` 的成员函数和运算符重载使得字符串的处理变得非常直观和方便。

这些特性使得 `std::string` 成为处理文本数据的首选工具，尤其在现代 C++ 编程中，是一个不可或缺类。